2025 3rd World Conference on Communication & Computing (WCONF)
Raipur, India. Jul 25-27, 2025

Design & Implementation of Sobel Edge Detection via Customized MAC IP in Vivado

Bharti Poddar[1]
School of VLSI & Embedded Systems
National Institute of Technology, Kurukshetra
Haryana, India
bhartipoddar55@gmail.com

Dr. Pankaj Verma[2]
Asst. Prof. Dept of ECE
National Institute of Technology, Kurukshetra
Haryana, India
pankaj@nitkkr.ac.in

Satyam Singh[3]
School of VLSI & Embedded Systems
National Institute of Technology, Kurukshetra
Haryana, India
satyamsingh275@gmail.com

2025 3rd World Conference on Communication & Computing (WCONF) | 979-8-3315-3045-7/25/$31.00 ©2025 IEEE | DOI: 10.1109/WCONF64849.2025.11233433

***Abstract*— This paper details the development and application of a custom Multiply-Accumulate (MAC) Intellectual Property (IP) core for implementing the Sobel edge detection algorithm, utilizing the Xilinx Vivado design suite. Edge detection is a vital process in digital image processing, with applications spanning medical imaging, autonomous vehicles, industrial inspection, and security systems, where precise boundary detection and feature extraction are critical. The Sobel operator is used to identify edges by computing the gradient magnitude of an image, emphasizing regions with significant intensity variations. To boost computational efficiency, this study introduces a hardware-centric approach by deploying the Sobel operator via a tailored MAC IP core. This custom IP core, designed for FPGA implementation, enhances the convolution operation by leveraging parallel processing to decrease computation time and latency. Evaluations conducted on the FPGA platform reveal notable enhancements in processing speed, energy consumption, and resource usage compared to conventional software methods. This strategy offers a powerful solution for real-time image processing tasks, such as video analysis, object detection, and autonomous navigation.**

Keywords*— *Sobel Edge Detection, MAC IP, Vivado, Image Processing.

I. INTRODUCTION

Image processing is an advanced technique extensively utilized in modern technological applications, involving the manipulation of imagery through sophisticated computational algorithms. As a specialized branch of signal processing, its primary input is an image, with the output potentially being a transformed image or extracted features. This area is pivotal within Computer Science and Engineering research, encompassing techniques such as visualization, feature recognition, enhancement, restoration, and pattern recognition. Image processing is divided into Analog Image Processing, which uses interpretative strategies for visual examination, and Digital Image Processing, which manipulates digital images using computers. A digital image is represented as a two-dimensional function, f(x, y), where x and y are spatial coordinates and f(x, y) signifies intensity at those points. Finite and discrete values make up digital images, composed of pixels with defined locations and intensity values. Digital image processing is crucial for preprocessing tasks in applications like facial recognition, object detection, and image compression, as it highlights key information based on specific application needs.

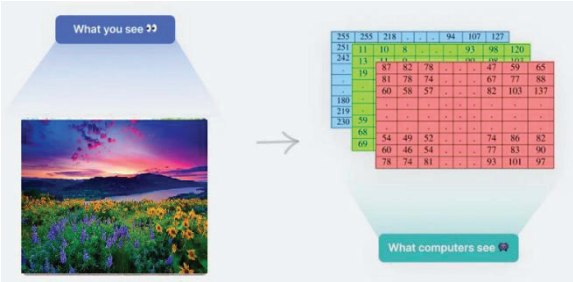


Fig. 1. Computer's Image Analysis.

A digital computer treats the digital image as a function: I(x, y) or I(x, y, z) where (x, y) or (x, y, z) are the coordinates of the pixel and I indicate the pixel intensity at a particular coordinate. Generally, we use 8-bit images, so the intensity ranges fr0m 0 to 255.

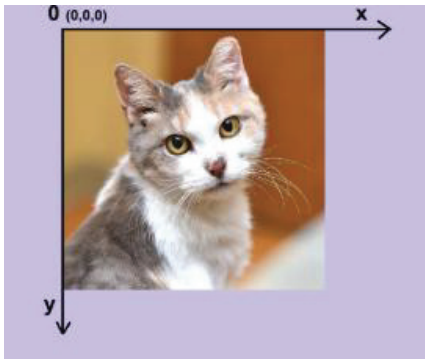


Fig. 2. Image coordinate system standard.

We utilize various types of images distinguished by their intensity values:

- **Binary Image:** These images consist of two intensity levels, 0 and 1, where 0 denotes black and 1 denotes white. Binary images are predominantly employed for object-background separation through segmentation.
- **Grayscale Image:** In an 8-bit grayscale image, there are 256 intensity levels ranging from 0 to 255. A value of 0 corresponds to black, 255 to white, and the values in between represent different shades of gray.
- **RGB Image:** Most everyday images are RGB, requiring three coordinates for each pixel, with values ranging from 0 to 255. The color black is indicated by (0, 0, 0), while white is represented by (255, 255, 255). Combinations of these values produce a broad spectrum of colors

Authorized licensed use limited to: XiangTan University. Downloaded on April 24,2026 at 06:53:40 UTC from IEEE Xplore. Restrictions apply.

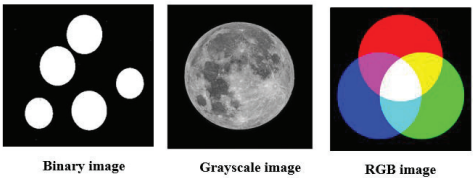


Fig. 3. Varieties of Image Formats.

Spatial Manipulations: Techniques executed directly on the image pixels are termed Spatial Manipulations. These are generally divided into two categories: Point or Single-pixel processing and Neighborhood processing. Such manipulations represent the most basic method applied to digital imagery, aimed at modifying pixel intensities through defined transformation functions, T, illustrated in the structure below. In point processing, the recalculated pixel value, g(x, y), is derived from the corresponding pixel value in f(x, y) alongside a specific operation.

$$g(x, y) = T[f(x, y)] \quad (1)$$

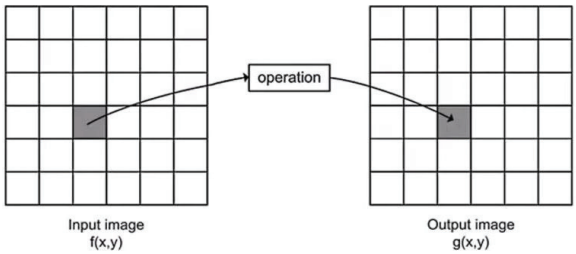


Fig. 4. Point Processing.

In neighborhood processing, the calculation of a new pixel value hinges not solely on the corresponding pixel intensity within the input image but also incorporates data from adjacent pixels around the target pixel. For an image denoted as 'f', the coordinates defining the neighborhood centered on a pixel (x, y) are represented by Sxy [5].

This approach facilitates the determination of a corresponding pixel in the output image g, utilizing the coordinate set Sxy. Consequently, the pixel value is influenced by a designated operation performed on its surrounding pixels.

$$g(x, y) = \text{operation}\left[\sum_{(r,c)\in S_{xy}} f(r, c)\right] \quad (2)$$

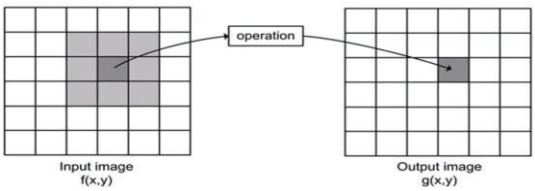


Fig. 5. Neighborhood Processing.

II. LITERATURE REVIEW

Reference [1], this paper investigates S0bel edge detection for advancing real-time image processing across multiple sectors, harnessing FPGA's parallelism through HDL synthesis. The design optimizes convolution kernels for gradient computation, focusing on throughput, latency, and resource utilization to affirm effectiveness.

Reference [2], this study addresses challenges in computational enhancement through approximate computing, presenting 8-bit multipliers with recursive, bit-wise, and hybrid approximation (PBO) techniques.

Compared to existing multipliers, these designs improve area, power, delay, PDP, and PADP by up t0 75.42%. Performance metrics PSNR and SSIM indicate robust application in image processing and deep neural networks, achieving over 95% efficiency, underscoring their suitability for error-resilient applications. Reference [7], addressing inaccuracies and high resource usage in conventional FPGA-based Sobel edge detection, this paper introduces an enhanced method utilizing an eight-direction Sobel operator and eight-bit architecture. This approach optimizes detection precision and minimizes hardware demands, reducing register resource consumption by 13.9% compared to traditional models. Reference [11], this paper introduces a VLSI architecture optimized for real-time video edge detection, leveraging parallel processing and refined data flow for enhanced speed. Employing Canny detection, the design emphasizes resource efficiency and memory management, implemented on Artix 7 and validated with GPP for performance assessment. Reference [12], this paper presents a Shift-Add Multiplication scheme for efficient RGB to Grayscale conversion employing NTSC coefficients on FPGAs. The designed module supports both fixed-point and floating-point formats, offering configurability in precision and resource optimization. A Python script facilitates automated Verilog code generation for fractional multiplications. Additionally, pipelined Gaussian Blur and Sobel modules enable a comprehensive real-time edge detection system. The findings demonstrate that Shift-Add multipliers markedly decrease dynamic power consumption relative to conventional FPGA DSP block usage.

III. SOBEL EDGE DETECTION ALGORITHM

Developed by Irwin Sobel and Gary Feldman at Stanford Artificial Intelligence Laboratory in 1968, the Sobel algorithm operates by moving a 3x3 matrix, known as a kernel, across the image. At each position, it calculates the

gradient change within the kernel. Notable variations in intensity signify the existence of an edge. The Sobel edge detection algorithm is a fundamental tool in image processing, widely utilized for its simplicity and effectiveness in highlighting edges within digital images. This paper describes an overview of the Sobel operator, detailing its computational approach using MAC IP and applications in various domains.

Edge detection is a critical step in image analysis, serving as a precursor to higher-level tasks such as object recognition and scene understanding. Among the various techniques available, the Sobel edge detection algorithm stands out due to its straightforward implementation and computational efficiency.

The Sobel operator employs two 3x3 convolution kernels to approximate the gradient of the image intensity function. These kernels are crafted to optimally detect edges that are oriented vertically and horizontally in relation to the pixel grid [1]. The horizontal kernel (Gx) and the vertical kernel (Gy) are defined as follows:

Authorized licensed use limited to: XiangTan University. Downloaded on April 24,2026 at 06:53:40 UTC from IEEE Xplore. Restrictions apply.

-1	0	+1
-2	0	+2
-1	0	+1

Gx

+1	+2	+1
0	0	0
-1	-2	-1

Gy

Fig. 6. Horizontal Kernel (Gx) Vertical Kernel (Gy).

By convolving these kernels with the input image, the algorithm computes the gradient magnitude and direction at each pixel. The gradient magnitude is given by:

$$|G| = \sqrt{Gx^2 + Gy^2} \tag{3}$$

Equation3, magnitude highlights regions of high intensity change, corresponding to edges in the image.

The algorithm extensively uses a mathematical operation called convolution. The convolution process captures significant changes in pixel intensity, which are indicative of edges. By adjusting the weights in the convolution kernel, the level of detail and quality of the detected edges can be controlled. This involves flipping the rows and columns of the kernel and then multiplying and summing the corresponding elements of the kernel and the image matrix. The standard Sobel operator uses two specific 3x3 matrices to detect changes along the x and y axes.

After preparing the convoluted matrices, the algorithm processes each pixel in the original image by applying the convolution kernels Gx and Gy. This operation produces a single value for each kernel.

$$\mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \mathbf{A}$$

Fig. 7. Pre-Computed Kernel (A matrix encapsulating the image data).

-1	0	+1
-2	0	+2
-1	0	+1

Gx

+1	+2	+1
0	0	0
-1	-2	-1

Gy

Fig. 8. Convoluted version of the above kernel.

The Sobel edge detection algorithm remains a popular choice in image processing due to its straightforward approach and effectiveness in edge detection tasks. While more advanced methods exist, the Sobel operator continues to be a valuable tool, particularly in applications where computational efficiency is paramount. The Sobel algorithm identifies edges by analyzing changes in pixel intensity within an image. This process is most straightforward when the image converted to a standardized grayscale format.

To create a grayscale image, one common method is to average the red, green, and blue (RGB) values of each pixel. This average is then used to set the RGB values to the same intensity, resulting in shade of gray (with #000000 representing black and #FFFFFF representing white). While averaging often yields satisfactory results, a more accurate approach involves using a weighted average that better reflects human visual perception of different colors and their transitions. Once a grayscale image is obtained, the Sobel edge detection algorithm can be applied.

IV. MAC IP

Edge detection is a vital task in image processing, identifying key intensity changes. The Sobel algorithm, known for its simplicity and effectiveness, can be enhanced with hardware acceleration to achieve real-time processing. Using Field-Programmable Gate Arrays (FPGAs) and the Vivado Design Suite, the Multiply-Accumulate (MAC) operation, crucial for convolution, can be efficiently implemented to boost the algorithm's performance. This paper discusses creating a MAC IP core with Vivado and integrating it into the Sobel algorithm for optimized performance. Vivado offers a strong platform for designing custom IP cores, with the MAC operation involving multiplication and accumulation being essential for the Sobel algorithm's convolution tasks.

The process includes defining design specifications, generating the IP core with Vivado's tools, simulating and verifying functionality, and synthesizing the design for FPGA implementation. The MAC IP core speeds up the Sobel algorithm by efficiently handling the multiply-accumulate operations needed for each pixel during kernel convolution. This hardware-based approach reduces processing time, enabling real-time edge detection even in high-resolution images.

Implementing the MAC IP in Vivado for the Sobel algorithm provides benefits such as real-time processing, efficient resource use, and scalability for different image sizes and resolutions.

This integration significantly enhances the Sobel algorithm's performance, offering a flexible and efficient solution for real-time image processing applications.

We can choose out of the 2 ways to design our system. One method is to use a separate IP of the multiplier and a separate IP of the adder, this design will occupy more space, and it is a little bit slow. Other methods consist of direct use of MAC IP which is already available as an inbuilt IP in Vivado. Design created using the second method will take up less space and be more efficient. Also, complexity is less in the second method.

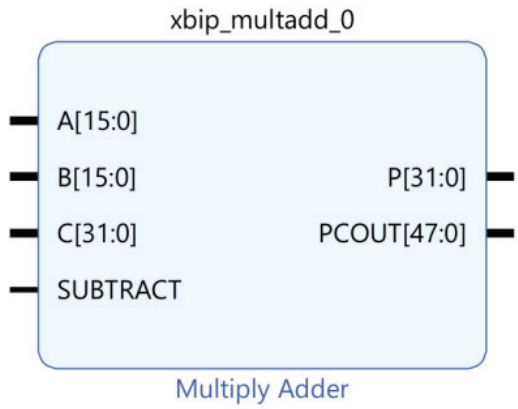


Fig. 9. MAC IP WRAPPER with 3 inputs.

Authorized licensed use limited to: XiangTan University. Downloaded on April 24,2026 at 06:53:40 UTC from IEEE Xplore. Restrictions apply.

Here, we will pass two input values. In our image, the grayscale pixels range from 0 to 255 in value. So, we need an 8-bit size register to store that value. So, the 'A' input will have a value of this grayscale pixel, and the 'B' input is one corresponding value of the kernel matrix. First multiplication will be done then the generated result will be of size 16-bit. So, we will perform this operation for the rest of the 8 values for both the matrices and add them one by one. One final output is generated and stored in the 'sum' variable. The below diagram will help to understand this process clearly.

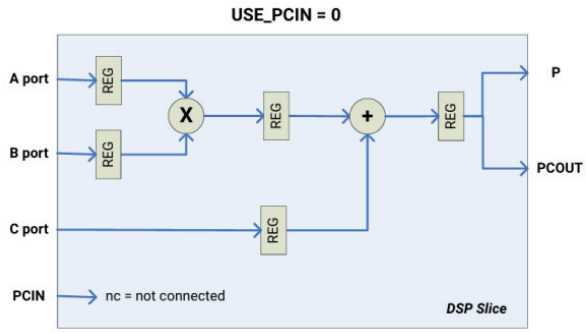


Fig. 10. MAC IP Internal Working.

I have designed the complete setup to do horizontal and vertical sum calculations. In the below diagram, green IPs are multipliers for the Gx calculation and pink IPs are for the Gy sum calculation. After multiplying the output will be given to adder IPs that are in orange color, and both the sum are generated. These 2 generated files are saved in the form of text files by using proper test bench simulation. Fig.11 shows the block diagram of MAC IP made in Vivado where A0-A8(each input of 32 bit) is image matrix, B0-B8 is kernel (each input of 32 bit), Input_0 is for 0 input.

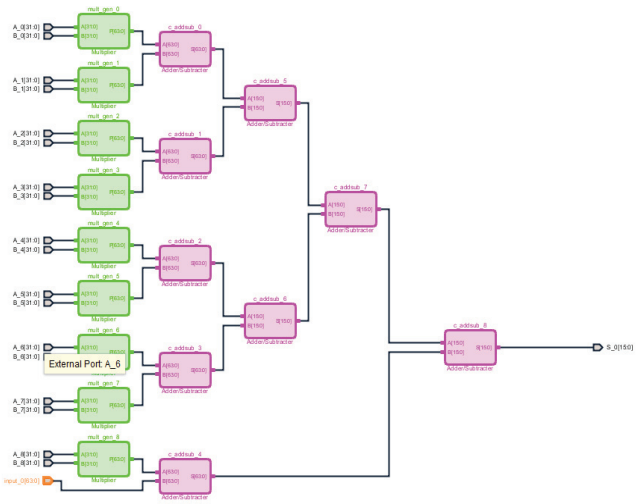


Fig. 11. MAC IP BLOCK DIAGRAM.

V. METHODOLOGY

A kernel is a small matrix used for image processing. Sobel Kernel is used here for edge detection. We must use 2 kernels, one for **horizontal** direction and the other for the **vertical** direction.

Zero Padding in convolutional operations enables the application of the kernel to the image's border pixels, helping to retain edge features that could otherwise be omitted. Without padding, the resulting image would be reduced in size compared to the input, potentially leading to the loss of critical edge information.

0	0	0	0	0
0	56	49	1	0
0	25	69	36	0
0	128	12	129	0
0	0	0	0	0

Fig. 12. Adding Layer of 0's.

Step 1: Conversion from RGB to Grayscale

The initial step in edge detection involves transforming a color image into a grayscale image. This is achieved using MATLAB, where the input RGB image is processed to eliminate color information, retaining only the intensity values. This step reduces the complexity of subsequent processing steps by focusing solely on luminance.

Step 2: Conversion from Grayscale to Binary

This process involves using MATLAB to create a binary image from the grayscale version, where each pixel is depicted as either black or white. This is done by applying a threshold: pixels above it are set to white, while those below are black, creating a binary image file.

Step 3: Quantization

Quantization involves converting the continuous range of pixel values into discrete levels. In this context, the binary image is quantized into an 8-bit format, which means each pixel is represented by a discrete value ranging from 0 to 255. This step is essential for digital processing, as it standardizes the data for further manipulation.

Step 4: Creating an Input File for the Binary Image

The quantized binary image is then prepared for further processing by creating an input file. This file is integrated into the Vivado IP through a simulation code, setting the stage for hardware-based processing.

Step 5: Convolution Using Multiplier and Adder Algorithm

The core of the edge detection process is the convolution operation, which is performed using a multiplier and adder algorithm. The image matrix undergoes convolution, where each pixel is multiplied by corresponding kernel weights and summed to produce a single output value. This operation is crucial for detecting changes in intensity, which signify edges. The gradients in the x and y directions, denoted as Gx and Gy, are computed and stored separately.

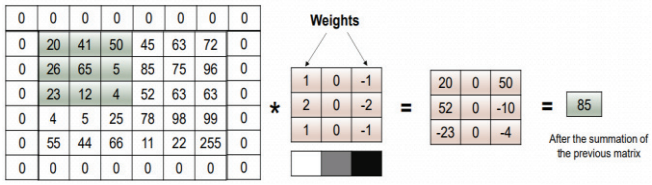


Fig. 13. Convolution Process.

Step 6: Generation of Output Binary File

Following the convolution process, a new binary file is generated. This file contains the results of the Gx and Gy calculations, representing the detected edges in binary form.

Authorized licensed use limited to: XiangTan University. Downloaded on April 24,2026 at 06:53:40 UTC from IEEE Xplore. Restrictions apply.

Step 7: Conversion from Binary to Numerical Format

The binary file is then converted into a numerical format, specifically an octal value file through MATLAB. This conversion facilitates further mathematical operations and analysis.

Step 8: Calculation of Gradient Magnitude

To assess the overall edge intensity, the gradient magnitude is determined by computing the square root of the sum of the squared values of Gx and Gy, as demonstrated by the formula:

$$G = \sqrt{Gx^2 + Gy^2}$$

This step provides a single value representing the edge intensity at each pixel.

Step 9: Saving the Processed Image

Finally, the processed image is saved in a standard image format, such as .jpg or .png. This step involves plotting the image data into a file format that can be easily viewed and analyzed, completing the edge detection process.

VI. RESULTS

In our work, we are using a sample photo of a cat whose dimensions are 386 pixels in height and 297 pixels in width. So, our input image is of the size 386 x 297 pixels. Each pixel consists of a gray scale value and here we used an 8-bit binary representation for each pixel. So, each pixel is represented using 8 bits. So, to get that bit file of the image, we need to process the file from MATLAB. After it, the binary image is processed in Vivado.

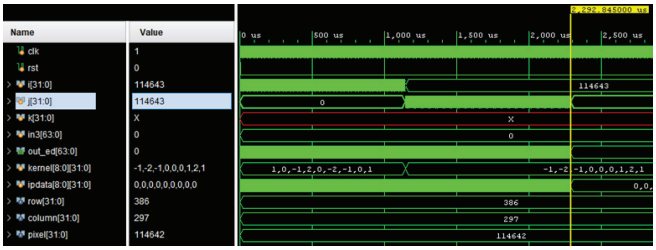


Fig. 14. Vivado output waveform

The simulation has been run for 11463 pixels 2 times, once for Gx & other for Gy image. **Total simulation time is 2.3ms**. this can be further reduced if the pixels are less.

The post-synthesis resource utilization report from the Xilinx Vivado tool shows that the design uses a minimal portion of the available FPGA resources. Only 5% of the Look-Up Tables (LUTs) are used for implementing combinational logic, while 6% of LUTRAM is utilized for distributed memory. The design requires just 1% of flip-flops, indicating limited use of sequential elements. Input/Output (I/O) resources account for 2%, reflecting a low number of external connections. Additionally, only 3% of global buffers (BUFGs) are used for clock distribution. This low resource consumption confirms that the design is efficient and leaves sufficient room for future enhancements or scaling.

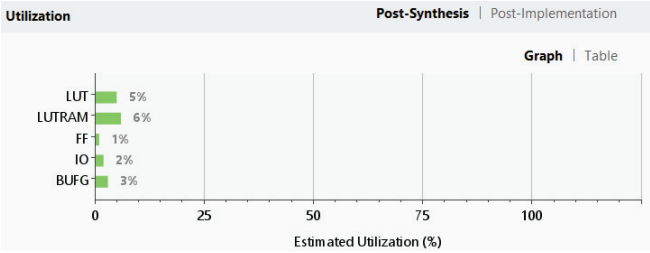
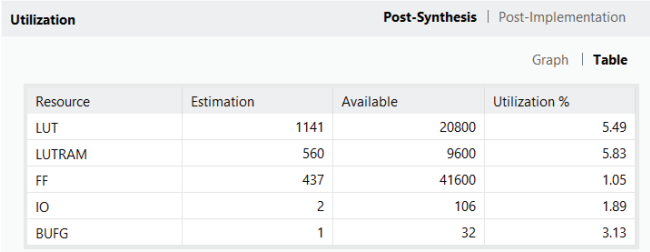


Fig. 15. Resource Utilization Snap from Vivado

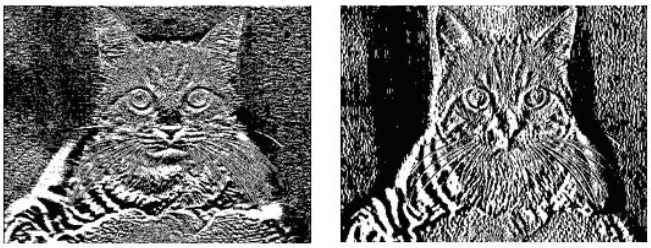


Utilization — Post-Synthesis | Post-Implementation — Graph | **Table**

Resource	Estimation	Available	Utilization %
LUT	1141	20800	5.49
LUTRAM	560	9600	5.83
FF	437	41600	1.05
IO	2	106	1.89
BUFG	1	32	3.13

Fig. 16. Utilization table

We have two text format files that have the results generated by Vivado. We input these files into MATLAB and again it will process and generate the corresponding output of each sum file. But for the generation of the final output, we need to merge these 2 output files into one by applying one formula to the pixel values obtained. the corresponding output image generated for horizontal and vertical sum files. These files alone can't be considered as perfect output.



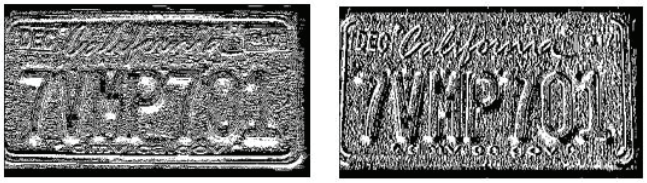
(a) (b)

Fig. 17. (a) Gx output image. (b) Gy output image.

(a) (b)

Fig. 18. (a) Original Image. (b) Final Output Image G.

One more Image is taken for sample of a license plate whose dimensions are- 360 x 203 pixels.



(a) (b)

Fig. 19. (a) Gx output image. (b) Gy output image.

Authorized licensed use limited to: XiangTan University. Downloaded on April 24,2026 at 06:53:40 UTC from IEEE Xplore. Restrictions apply.

(a) (b)

Fig. 20. (a) Original Image. (b) Final Output Image G.

VII. CONCLUSIONS & FUTURE SCOPE

In this paper, we have introduced an innovative implementation of the Sobel edge detection algorithm utilizing a customized Multiply-Accumulate (MAC) IP core, illustrating marked improvements in computational efficiency and throughput. Unlike traditional FPGA implementations, our method leverages the Vivado Design Suite to finely optimize the MAC operations central to the convolution process, significantly enhancing real-time processing capabilities. The tailored MAC IP not only accelerates the algorithm but also upholds superior accuracy in edge detection, even with high-resolution data. This underscores the advantage of MAC IP solutions over conventional FPGA approaches, offering a streamlined, resource-efficient path to superior image processing performance. Future directions might involve integrating this MAC IP with other algorithms, such as Canny or Prewitt, to ascertain its versatility. Additionally, scaling this MAC IP for advanced tasks like object recognition and image segmentation could broaden its applicability. Exploration into embedding this technology into IoT devices, where power efficiency is paramount, or further refining it with advances in FPGA technology, could lead to groundbreaking enhancements in speed and resource utilization.

REFERENCES

[1] V. Priyanka, Y. S. Rama, K. Sravani and B. Kavya, "Implementation of Sobel Edge Detection with Image Processing on FPGA," *2024 2nd World Conference on Communication & Computing (WCONF)*, RAIPUR, India, 2024, pp. 1-5.

[2] A. Kumari and R. P. Palathinkal, "Design and Analysis of Energy Efficient Approximate Multipliers for Image Processing and Deep Neural Network," in *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, no. 2, pp. 854-867, Feb. 2025.

[3] B. Atef, R. Abdelhalim, S. Abdelkerim and Y. Djeghader, "FPGA Implementation of Flexible Architecture for Image Processing," *2024 International Conference on Telecommunications and Intelligent Systems (ICTIS)*, Djelfa, Algeria, 2024, pp. 1-5.

[4] T. I. Haq, M. M. Mafazy, J. T. Agustinus, N. J. de La Croix and T. Ahmad, "Sobel Edges Key-Based Method for Steganography in Spatial Domain Images," *2023 3rd International Conference on Mobile Networks and Wireless Communications (ICMNWC)*, Tumkur, India, 2023, pp. 1-5.

[5] T. Selarka, Y. Viradiya and D. Shah, "Neighborhood Image Processing Using Verilog HDL," *2023 3rd International Conference on Advancement in Electronics & Communication Engineering (AECE)*, GHAZIABAD, India, 2023, pp. 494-498.

[6] S. Kumar, A. K. Upadhyay, P. Dubey and S. Varshney, "Comparative analysis for Edge Detection Techniques," *2021 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, Greater Noida, India, 2021, pp. 675-681

[7] Z. Liu, Y. Wang, P. Chen, F. Wang and H. Zhang, "Implementation of Eight-Bit Architecture Sobel Algorithm Based on FPGA," *2024 IEEE 7th International Conference on Automation, Electronics and Electrical Engineering (AUTEEE)*, Shenyang, China, 2024, pp. 547-550.

[8] N. Kaur, K. Sharma and A. Jain, "Hybrid Edge Detection Method for Autonomous Vehicle Navigation," *2024 7th International Conference on Contemporary Computing and Informatics (IC3I)*, Greater Noida, India, 2024, pp. 167-172.

[9] L. Shi and Y. Zhao, "Edge Detection of High-Resolution Remote Sensing Image Based on Multi-Directional Improved Sobel Operator," in *IEEE Access*, vol. 11, pp. 135979-135993, 2023.

[10] W. Zhang and W. Ji, "Real-Time License Plate Positioning Algorithm Optimization and Hardware Implementation," *2024 5th International Conference on Computer Engineering and Intelligent Control (ICCEIC)*, Guangzhou, China, 2024, pp. 140-143.

[11] V. G. R. Maddipati, B. Soudha Basheera Bhanu, D. Bhavya Sai, G. Navya Sri Mahitha, G. Pravallika and G. Maddipati, "High Performance VLSI Architecture for Real-Time Video Edge Detection," *2023 Second International Conference on Advances in Computational Intelligence and Communication (ICACIC)*, Puducherry, India, 2023, pp. 1-6.

[12] N. Rout and J. Jean Jenifer Nesam, "Optimizing RGB to Grayscale, Gaussian Blur and Sobel-Filter operations on FPGAs for reduced dynamic power consumption," *2024 3rd International Conference on Artificial Intelligence For Internet of Things (AIIoT)*, Vellore, India, 2024, pp. 1-6.

Authorized licensed use limited to: XiangTan University. Downloaded on April 24,2026 at 06:53:40 UTC from IEEE Xplore. Restrictions apply.